Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: _____Nicolle Escobar_____ **Carné**: ___1264425_____

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado
Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:
GitHub: https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet
El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

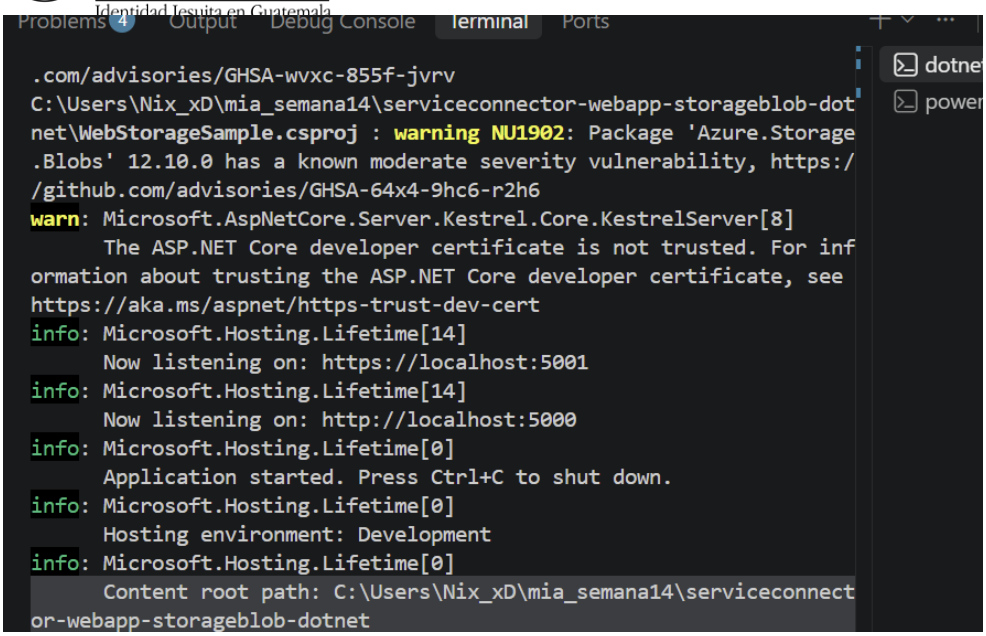
Realización de la actividad

1. Preparar el repositorio
En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

---

2. Revisar la aplicación:
Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

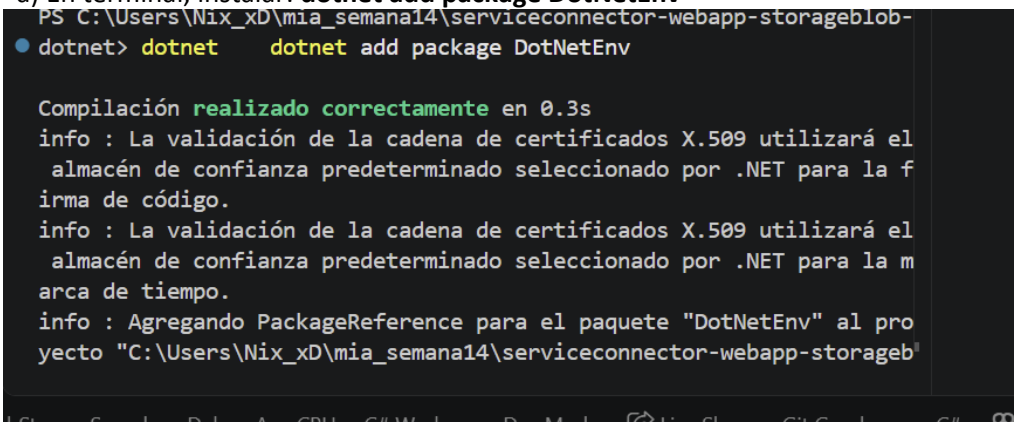
NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>



---

3. Crear DotNet Env

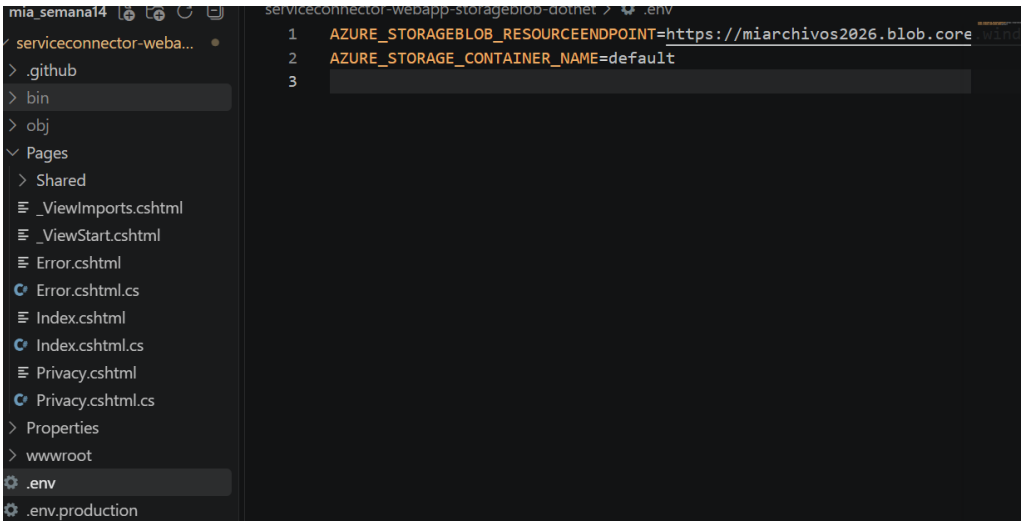
a) En terminal, instalar: **dotnet add package DotNetEnv**



b) Crear .env file en raíz de proyecto, valores ->

AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/

AZURE_STORAGE_CONTAINER_NAME=default



c) Crear .env.production con los mismos valores en el mismo lugar
d) Anadir cambio brindado en program.cs
e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run

```
PS C:\Users\Nix_xD\mia_semana14\serviceconnector-webapp-storageblob-dotnet> dotnet run
da conocida, https://github.com/advisories/GHSA-wvxc-855f-jvrv
C:\Users\Nix_xD\mia_semana14\serviceconnector-webapp-storageblob-dotnet\WebStorageSample.csproj : warning NU1902: El paquete "Azure.Storage.Blobs" 12.10.0 tiene una vulnerabilidad de gravedad
 moderada conocida, https://github.com/advisories/GHSA-64x4-9hc6-r2h6
warn: Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServer[8]
      The ASP.NET Core developer certificate is not trusted. For information about trusting the ASP.NET Core developer certificate, see https://aka.ms/aspnet/https-trust-dev-cert
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\Nix_xD\mia_semana14\serviceconnector-webapp-storageblob-dotnet
info: Microsoft.Hosting.Lifetime[0]
      Application is shutting down...
```

4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

a) Azure Portal → **Storage accounts → tu Storage Account → Data storage → Containers**
b) Crear contenedor “default
c) Activa identidad administrada en App Service: **Azure Portal → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)**
d) Configurar la conexión entre App Service y Blob Storage: **App Service → Settings → Service Connector → Create → Seleccionar “Storage Blob” → Seleccionar Storage Account [tu_blob_name] → Authentication “System-assigned managed identity” → Review + Create**.
e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service



6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

a) Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: az --version

b) Inicie sesión en Azure: az login
c) Verifique la cuenta de Azure activa: az account show
d) Verifique que pueda obtener un token para Azure Storage: az account get-access-token --resource https://storage.azure.com/
e) Si se muestra un accessToken la autenticación esta correcta, ejecutar -> dotnet run
f) Si hay error en acceso, ejecutar en terminarl vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table
az storage account show --name “tu_storage” --query "{id:id, resourceGroup:resourceGroup}" -o table
g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run

Nota: no olvidar subir cambios a repo github fork

---

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo “azure-pipelines.yml”
Colocar en el archivo lo siguiente:

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

  # Completar posteriormente
  azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
  appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

# 1. Instalar .NET 9
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'

# 2. Restaurar dependencias
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
```

```
    projects: '**/*.csproj'

# 3. Configurar ambiente de producción
- powershell: |
    Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'

# 4. Compilar
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'




# 5. Publicar
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-
build'
    zipAfterPublish: true

# 6. Desplegar en Azure App Service
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

---

8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

a) Conectarse a Azure Devops son misma cuenta que azure portal -> https://aex.dev.azure.com/me

b) Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**

c) Probar seleccionando “run”

d) Conectar Azure DevOps con Azure: **Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre “MIA_Connection” -> Seleccionar “grant access permission to all pipelines” -> Save**

8. Conectar Pipeline con App

a) Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app

b) en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name
appName: Tu _app_name

c) Al hacer commit, verificar el pipeline se ejecuta automáticamente

9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

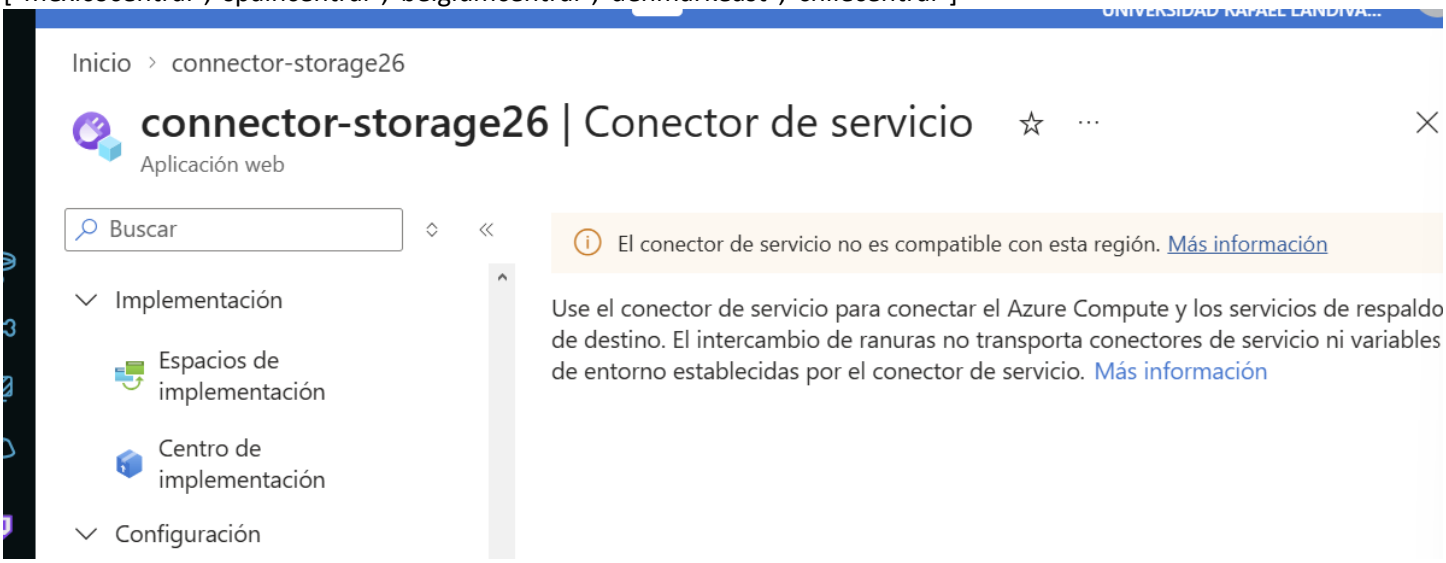
a) acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings -> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)

b) Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica

c) En la app publicada realizar cambios en archivos

d) Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios

Estas son las regiones que tiene permitida mi cuenta:
["mexicocentral","spaincentral","belgiumcentral","denmarkeast","chilecentral"]



Inte con una cuenta que no era de la universidad pero me pedí meter una tarjeta de crédito.

10. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?
   Se utiliza para almacenar variables de entorno y credenciales sensibles y se debe ver solo para el equipo, ya que el env.production es para uso online.

2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
   Porque los servidores de aplicaciones como azure pueden reiniciarse, actualizarse o escalar horizontalmente en múltiples instancias.

3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
   - URL de App Service: Es la dirección web donde se ejecuta el backend o la interfaz de tu aplicación.
   - Endpoint de Blob Storage: Es la dirección directa al servicio de almacenamiento en la nube, optimizada exclusivamente para guardar, administrar y servir archivos binarios.

4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
   Por la seguridad ya que elimina el almacenar clave de acceso estáticas, archivos de configuración o variables de entorno. La otra es por la gestión automática.

5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
   - GitHub: Es el repositorio de control de versiones.
   - Azure DevOps: Es la plataforma de CI/CD que automatiza los procesos de compilación, ejecución de pruebas y despliegue del software.
   - Service Connection: Es el puente seguro de autenticación que permite a Azure DevOps conectarse a una suscripción de Azure para desplegar recursos sin necesidad de escribir contraseñas fijas en los pipelines.

6. ¿Qué ocurre después de hacer push a main?
   Se verifica que el código este actualizado, que compile, en general que funcione para poder mandar la versión actualizada del main.

7. Si el pipeline termina correctamente, pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?
   Reviso la identidad administrativa de la aplicación que asigna los roles ya que el error 403 es que el cliente no tiene los permisos necesarios para realizar acciones.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.
Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.
Enviar link de github a plataforma de la universidad.